



SIMULADO PARA PROVA 01

Nome do aluno: _____ Prontuário: _____

QUESTÃO 1 (2,0 pontos) Considerando o programa em Linguagem C a seguir, identifique as funções de ordenação correspondentes aos respectivos algoritmos de ordenação

Algoritmo de ordenação	Nome da função neste programa
<i>Bubble sort</i>	
<i>Insertion sort</i>	
<i>Selection sort</i>	
<i>Merge sort</i>	

Além disso responda: por qual motivo foi chamada na função main a função **F2 (nords, TAMANHO+3)** ; e não outra?

RESPOSTA: _____

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define TAMANHO 10
void F0 (char nomes[][50], int T){
    int i;
    for (i = 0; i < T; i++)
        printf("%s\n", nomes[i]);
}
void F1(char nomes[][50], int T) {
    int i, j;
    char temp[50];
    for (i = 0; i < T - 1; i++) {
        for (j = 0; j < T - i - 1; j++) {
            if (strcmp(nomes[j], nomes[j + 1]) > 0) {
                strcpy(temp, nomes[j]);
                strcpy(nomes[j], nomes[j + 1]);
                strcpy(nomes[j + 1], temp);
            }
        }
    }
}
void F2(char nomes[][50], int T) {
    char escolhido[50];
    int anterior, i;
    for (i = 1; i < T; i++) {
        strcpy(escolhido, nomes[i]);
        anterior = i - 1;
        while ( (anterior >= 0) && ( strcmp(nomes[anterior], escolhido ) > 0 ) ) {
            strcpy(nomes[anterior + 1], nomes[anterior]);
            anterior--;
        }
        strcpy(nomes[anterior + 1] , escolhido);
    }
}
void F3 (char nomes[][50], int T) {
    int pm, i, j;
    char aux[50];
    for(i=0; i < T-1; i++) {
        pm = i;
        for (j=i+1; j < T; j++) {
            if ( strcmp(nomes[j], nomes[pm] ) < 0 )
                pm = j;
        }
        if (pm != i) {
            strcpy(aux , nomes[i]);
            strcpy(nomes[i] , nomes[pm]);
            strcpy(nomes[pm], aux);
        }
    }
}
```

```

int main() {
    char nomes[TAMANHO][50] = {"Carlos", "Ana", "Zeca", "Bia", "Davi", "Eva", "Gabriel", "Hugo", "Isabela",
    "Jorge"};
    char nords[TAMANHO+3][50] = {"", "", "", "", "", "", "", "", "", "", "", "", "", "", ""};
    int i;
    printf("Nomes antes de ordenar:\n");
    F0(nomes, TAMANHO);
    F1(nomes, TAMANHO);
    printf("\nNomes depois de ordenar:\n");
    F0(nomes, TAMANHO);
    for ( i = 0; i < TAMANHO; i++) strcpy(nords[i], nomes[i]);
    printf ("\nDigite o 1lo nome: "); fflush (stdin); gets(nords[10]);
    printf ("\nDigite o 12o nome: "); fflush (stdin); gets(nords[11]);
    printf ("\nDigite o 13o nome: "); fflush (stdin); gets(nords[12]);
    printf("\nNomes depois de completar:\n");
    F0(nords, TAMANHO+3);
    /* Chama a função de ordenação mais recomendável para esta situação: */
    F2(nords, TAMANHO+3);
    printf("\nNomes depois de ordenar:\n");
    F0(nords, TAMANHO+3);
    return 0;
}

```

QUESTÃO 2 (2,0 pontos) Considerando o programa em Linguagem C a seguir, recodifique a função **encontrarMinimo** de modo que ela passe a ser recursiva. A função **encontrarMinimo** recebe um vetor e seu tamanho e procura pelo menor elemento contido no vetor. Abaixo segue exemplo de funcionamento de um programa que utiliza a função solicitada.

```

C:\IFSP\000_AULAS_IFSP_202 x + v - □ x
[3,5,75,2,8]
O valor mínimo no array é: 2
-----
Process exited after 0.03857 seconds with return value 0
Pressione qualquer tecla para continuar. . . |

```

```

#include <stdio.h>
#include <locale.h>

int encontrarMinimo(int *vet, int tamanho) {
    int minimo = vet[0], i;
    for (i = 1; i < tamanho; i++) {
        if (vet[i] < minimo) {
            minimo = vet[i];
        }
    }
    return minimo;
}

void mostraArray(int * vet, int tamanho){
    int i;
    printf("\n[");
    for (i = 0; i < tamanho; i++)
        if (i<tamanho-1)
            printf("%i,", vet[i]);
        else
            printf("%i]\n", vet[i]);
}

int main() {
    int arr[] = {3, 5, 75, 2, 8}, tamanho;
    setlocale(LC_ALL, "");
    tamanho = sizeof(arr) / sizeof(arr[0]);
    mostraArray(arr, tamanho);
    printf("\nO valor mínimo no array é: %d\n", encontrarMinimo(arr, tamanho));
    return 0;
}

```

RESPOSTA:

QUESTÃO 3 (2,0 pontos) Quantas vezes a função **Fx** é autochamada quando **b=12** e **e=8**?

```
int Fx(unsigned long long int b, unsigned long long int e) {  
    if (e == 0)  
        return 1;  
    else  
        return b * Fx(b, e - 1);  
}
```

RESPOSTA:

QUESTÃO 4 (2,0 pontos) Para cada afirmativa a seguir, assinale verdadeiro ou falso, sendo que cada resposta errada anula uma certa.

	Afirmativa	V/F
1	Bubble Sort é um algoritmo de ordenação simples, sendo o mais eficiente para vetores com grande quantidade de elementos devido à sua complexidade de tempo $O(n^2)$.	
2	No algoritmo de Insertion Sort , os elementos de um vetor são ordenados de forma crescente ao inserir cada elemento do vetor na sua posição correta, comparando-o apenas com o primeiro elemento do vetor.	
3	O Selection Sort funciona selecionando o menor elemento de um vetor a cada passo e colocando-o na posição correta (começo ou final do vetor, a depender da ordenação desejada – crescente ou decrescente), repetindo até que todo o vetor esteja ordenado.	
4	O Merge Sort é um algoritmo de ordenação que utiliza a estratégia de divisão e conquista, dividindo a lista em partes menores, ordenando-as e, em seguida, combinando-as para obter a lista ordenada.	
5	Considerando o vetor {"São Paulo", "Curitiba", "Porto Alegre", "Brasília", "Natal"}, todos os estados intermediários desse vetor na memória serão: 1. {"São Paulo", "Curitiba", "Porto Alegre", "Brasília", "Natal"} (Início) 2. {"Curitiba", "Curitiba", "Porto Alegre", "Brasília", "Natal"} 3. {"Curitiba", "São Paulo", "Porto Alegre", "Brasília", "Natal"} 4. {"Curitiba", "Porto Alegre", "Porto Alegre", "Brasília", "Natal"} 5. {"Curitiba", "Porto Alegre", "São Paulo", "Brasília", "Natal"} 6. {"Brasília", "Porto Alegre", "São Paulo", "Brasília", "Natal"} 7. {"Brasília", "Curitiba", "São Paulo", "Brasília", "Natal"} 8. {"Brasília", "Curitiba", "Natal", "Brasília", "Natal"} 9. {"Brasília", "Curitiba", "Natal", "Porto Alegre", "Natal"} 10. {"Brasília", "Curitiba", "Natal", "Porto Alegre", "São Paulo"} (Fim)	

QUESTÃO 5 (2,0 pontos) Analise o código em Linguagem C a seguir.

```
#include <stdio.h>

void xprint (int * array, int size){
    int i;
    printf("\n\n[");
    for (i=0; i<size; i++)
        if (i==size-1)
            printf("%i]", array[i]);
        else printf("%i,", array[i]);
}

void xsort(int *array, int size) {
    int step, key, j;
    for (step = 1; step < size; step++) {
        key = array[step];
        j = step - 1;
        while (j >= 0 && key < array[j]) {
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = key;
        xprint(array, 5);
    }
}

int main(){
    int dados[5] = { 2, 5, 1, 4, 3 }, i;
    xsort(dados, 5);
    return 0;
}
```

Considere a seguinte saída do código apresentado:

[2, 5, 1, 4, 3]
[1, 2, 5, 4, 3]
[1, 2, 4, 5, 3]
[1, 2, 3, 4, 5]

O algoritmo de ordenação implementado no código apresentado é o:

- ☐ A) Heap Sort.
- ☐ B) Merge Sort.
- ☐ C) Bubble Sort.
- ☐ D) Insertion Sort.
- ☐ E) Selection Sort.

RESPOSTA: ____

QUESTÃO 6 (BÔNUS-2,0 pontos) Elabore o programa C que contenha uma função que realize a pesquisa binária no seguinte vetor:

{"Brasília", "Curitiba", "Natal", "Porto Alegre", "São Paulo"}